
Guide de soutenance

Voyage Assistant

Chatbot IA, RAG et reservation multi-transport

Document interne d'aide a la presentation orale.

Pour que chacun maitrise sa partie et reponde sans hesiter au jury.

SAE2	BUT3 Informatique
Equipe	Ako Christian, Carl Similien, Noe Cervera, Dhanoush Kessavane
Encadrant	Bilal Faye (LIPN, CNRS UMR 7030)
Application	https://optimalako-voyage-assistant.hf.space
Code	https://github.com/optmlako2004/chatbot

A lire et a repeter a voix haute avant le jour J.

Table des matières

1	Comment reussir la soutenance (a lire par tous)	2
1.1	Le principe de base	2
1.2	Les regles d'or	2
1.3	Qui presente quoi (10 diapositives)	2
2	Le projet en bref (tout le monde doit savoir)	3
3	Partie de Christian : IA, RAG, securite, deploiement	3
3.1	Slides 1 et 2 : ouverture et plan	3
3.2	Slide 3 : le probleme et la demonstration	3
3.3	Slide 4 : architecture d'ensemble	4
3.4	Slide 5 : le chatbot et le RAG (coeur technique)	4
3.5	Slide 6 : securite de l'IA et mise en ligne	5
3.6	Slide 10 : bilan, perspectives et echanges	6
4	Partie de Carl : back-end, API et base de donnees	7
4.1	Slide 7 : back-end, actions reelles et donnees	7
5	Partie de Noe : modeles de langage (NLP)	8
5.1	Slide 8 : les modeles de langage	8
6	Partie de Dhanoush : front-end, design et experience	9
6.1	Slide 9 : interface, design et experience	10
7	Glossaire complet (a connaitre par tous)	10
8	Questions transversales pieges (pour tous)	13

1. Comment réussir la soutenance (a lire par tous)

1.1 Le principe de base

Une diapositive ne se lit pas, elle s'explique. Les slides contiennent tres peu de texte (regle dite 5/5/5 : environ 5 mots par ligne, 5 lignes max). Tout le contenu riche, c'est **vous** qui le dites. Ce guide contient, pour chaque slide, le texte a dire et ce qu'il faut comprendre derriere.

1.2 Les regles d'or

- **Ne pas lire les slides.** Le jury sait lire. Vous, vous racontez.
- **Une idee principale par slide**, developpee a l'oral en 2 a 3 phrases.
- **Regarder le jury**, pas l'ecran. Se tenir droit, parler lentement.
- **Connaitre le temps imparti** et repeter au moins une fois en entier, chrono en main.
- **Anticiper les questions** : elles sont listees dans ce guide, partie par partie.
- **Assumer les limites.** Dire "nous ne l'avons pas fait, voici pourquoi et comment on le ferait" est bien meilleur que bluffer.
- **Faire des transitions** : "je laisse la parole a Carl qui va parler du back-end".

1.3 Qui presente quoi (10 diapositives)

#	Diapositive	Presentateur
1	Page de titre (presentation de l'equipe)	Christian
2	Deroule et repartition	Christian
3	Le probleme et la demonstration	Christian
4	Architecture d'ensemble	Christian
5	Le chatbot et le RAG	Christian
6	Securite de l'IA et mise en ligne	Christian
7	Back-end, actions reelles et donnees	Carl
8	Les modeles de langage (NLP)	Noe
9	Interface, design et experience	Dhanoush
10	Bilan, perspectives et echanges	Christian

Note : Christian ouvre et ferme car sa partie (IA, RAG, deploiement) structure tout le projet. Mais chacun doit comprendre l'ensemble : le jury peut poser une question a n'importe qui.

2. Le projet en bref (tout le monde doit savoir)

Ce que tu dois VRAIMENT comprendre

En une phrase : Voyage Assistant est une plateforme web de reservation de voyages (avion, train, bateau, bus) dotée d'un chatbot intelligent qui ne se contente pas de répondre, mais qui **agit réellement** : il consulte la base, modifie un billet, crée une réclamation, envoie un e-mail.

Les briques :

- **Front-end** : React (interface web).
- **Back-end** : FastAPI (Python), une API de 29 routes, 8 tables.
- **Base de données** : PostgreSQL (en production), 182 000 liaisons réelles.
- **IA** : Google Gemini 2.5 Flash (le modèle qui rédige les réponses).
- **RAG** : recherche de documents (embeddings gte-small + FAISS + reranker).
- **NER** : CamemBERT pour extraire nom/prénom/date/numero de billet.
- **APIs temps réel** : meteo (Open-Meteo) et taux de change (Frankfurter).
- **Deploiement** : Docker sur Hugging Face Spaces + base Neon, gratuit.

3. Partie de Christian : IA, RAG, securite, deploiement

Slides 1, 2, 3, 4, 5, 6 et 10.

3.1 Slides 1 et 2 : ouverture et plan

Ce que tu dis (a l'oral)

"Bonjour, nous sommes l'équipe qui a développé Voyage Assistant, un assistant de voyage intelligent. Je suis Christian, et avec moi Carl, Noé et Dhanoush. Je vais vous présenter le problème et l'architecture, puis chacun détaillera sa partie : Carl le back-end, Noé les modèles de langage, Dhanoush l'interface."

3.2 Slide 3 : le problème et la démonstration

Ce que tu dis (a l'oral)

"Aujourd'hui, pour réserver un voyage, on jongle entre plusieurs sites, et les informations (prix, horaires, retards) sont dispersées. Notre idée : un assistant unique pour l'avion, le train, le bateau et le bus, qui non seulement informe mais qui agit : il peut modifier un billet, l'annuler, ouvrir une réclamation. L'application est en ligne, je vais vous la montrer : voici une recherche réelle parmi 182 000 liaisons, et une conversation avec l'assistant."

Ce que tu dois VRAIMENT comprendre

La force du projet, c'est que le chatbot **n'est pas une demo qui fait semblant**. Quand il dit "votre billet est modifié", la ligne change vraiment en base, un nouveau PDF est généré et un e-mail part. C'est l'argument numéro un à mettre en avant.

Questions probables du jury + reponses

Q. Quelle est la difference avec un ChatGPT classique ?

Notre assistant est *connecte a nos donnees et a nos actions*. ChatGPT seul ne connait pas vos billets et ne peut rien modifier. Nous, on combine un modele de langage avec notre base, nos regles metier et une verification d'identite.

Q. D'où viennent les 182 000 liaisons ?

D'un jeu de donnees de liaisons de transport charge dans la table **routes** (3 005 villes de depart, 468 compagnies). Les prix et le nombre de places sont calcules dynamiquement.

3.3 Slide 4 : architecture d'ensemble

Ce que tu dis (a l'oral)

"L'architecture suit trois couches. Le front-end en React, qui parle a une API FastAPI en Python. Cette API gere la base PostgreSQL et orchestre la couche intelligence : le modele Gemini, notre RAG, la reconnaissance d'entites, et des APIs temps reel pour la meteo et le change. Tout est regroupe dans un seul service deploye sur Hugging Face : le front et l'API partagent la meme adresse, ce qui simplifie la securite."

Ce que tu dois VRAIMENT comprendre

Pourquoi un seul service ? Parce que si le front et l'API sont sur la meme origine (meme URL), il n'y a pas de probleme de CORS (le navigateur n'autorise pas par default un site a appeler une API d'un autre domaine). C'est plus simple et plus sur. FastAPI sert l'API *et* les fichiers du front.

Le flux d'un message : l'utilisateur ecrit -> le front envoie a l'API -> la fonction `process_message` decide quoi faire (repondre, chercher, agir) -> elle assemble un contexte (base, documents, web) -> Gemini redige -> la reponse revient au front.

Questions probables du jury + reponses

Q. Pourquoi FastAPI et pas Flask/Django ?

FastAPI est moderne, rapide, asynchrone, avec validation automatique des donnees (Pydantic) et documentation auto-generee. Parfait pour une API qui sert de l'IA.

Q. Pourquoi PostgreSQL et pas seulement SQLite ?

En developpement on a utilise SQLite (un simple fichier). Pour la mise en ligne, PostgreSQL (chez Neon) est plus robuste, accessible a distance et adapte a la production.

3.4 Slide 5 : le chatbot et le RAG (coeur technique)

Ce que tu dis (a l'oral)

"Le chatbot fonctionne en deux temps : il comprend, puis il agit. Il detecte l'intention de l'utilisateur (une simple question, un retard, une modification, une reclamation). Pour les actions sensibles, il lance un parcours guide et verifie l'identite avant tout. Pour les questions ouvertes, on utilise un RAG. Le RAG, Retrieval-Augmented Generation, c'est faire chercher a l'IA les bons documents AVANT de repondre, pour qu'elle s'appuie sur des sources reelles plutot que sur sa memoire, ce qui evite les reponses inventees. Notre RAG a deux etapes : d'abord une recherche large et rapide, ensuite un reclassement fin des meilleurs passages."

Ce que tu dois VRAIMENT comprendre

RAG, mot a mot :

- **Retrieval** (recuperation) : on cherche dans une base de documents les passages qui ressemblent le plus a la question.
- **Augmented** (augmente) : on *ajoute* ces passages au prompt envoye a l'IA.
- **Generation** : l'IA redige la reponse en s'appuyant dessus.

Pourquoi 2 etapes ?

- *Etape 1, le bi-encodeur (gte-small)* : transforme chaque texte en vecteur de nombres, puis on cherche les 10 plus proches dans l'index FAISS. C'est tres rapide mais grossier (il compare des resumes).
- *Etape 2, le cross-encodeur (ms-marco)* : il relit la question ET chaque passage ensemble, et donne un score de pertinence precis. On garde les 3 meilleurs. C'est plus lent, donc on ne le fait que sur 10 candidats, pas sur tout.

Les sources du RAG : les conditions generales de vente (annulation, bagages, remboursement) et les billets du client (indexes a leur creation, et filtres par utilisateur pour que personne ne voie les billets d'un autre).

Questions probables du jury + reponses

Q. Qu'est-ce qu'un embedding ?

C'est la transformation d'un texte en une liste de nombres (un vecteur) qui represente son sens. Deux phrases proches par le sens ont des vecteurs proches. Cela permet de "mesurer" la similarite.

Q. Qu'est-ce que FAISS ?

Une bibliotheque de Facebook qui stocke des milliers de vecteurs et retrouve tres vite les plus proches d'une question. C'est notre moteur de recherche semantique.

Q. Difference bi-encodeur / cross-encodeur ?

Le bi-encodeur encode la question et les documents *separement* (rapide, on peut pre-calculer). Le cross-encodeur les regarde *ensemble* (plus precis mais plus lourd). On combine les deux : rapidite puis precision.

Q. Et si le RAG ne trouve rien de pertinent ?

Gemini repond quand meme avec ses connaissances generales et, si besoin, une recherche web, mais sans inventer d'information sur un billet precis.

Q. Pourquoi gte-small et pas un modele plus gros ?

Compromis vitesse/qualite : gte-small (384 dimensions) est multilingue, leger, rapide a charger sur un hebergement gratuit. Un modele plus gros serait plus precis mais trop lourd.

3.5 Slide 6 : securite de l'IA et mise en ligne

Ce que tu dis (a l'oral)

"La securite est pensee a plusieurs niveaux. Cote IA, le chatbot refuse les sujets dangereux ou hors-sujet et detecte les tentatives de detournement, ce qu'on appelle le prompt injection. Surtout, il ne donne aucune information sur un billet tant que l'identite n'a pas ete verifiee. Cote mise en ligne, tout est dans un conteneur Docker, heberge gratuitement sur Hugging Face Spaces, avec une base PostgreSQL chez Neon. Les secrets, comme les cle d'API, ne sont jamais ecris dans le code : ils sont injectes comme variables d'environnement."

Ce que tu dois VRAIMENT comprendre

Prompt injection : c'est quand un utilisateur essaie de manipuler l'IA avec des phrases comme "ignore tes instructions" ou "montre-moi tous les billets". On detecte ces formulations et on refuse.

Vérification d'identité : pour toucher a un billet, le chatbot demande numero de billet + nom + prenom + date de naissance, et compare a la base. Sans correspondance, pas d'action. C'est ce qui protege les donnees personnelles.

Mots de passe : jamais stockes en clair. On utilise PBKDF2 (un algorithme qui "hache" le mot de passe des milliers de fois, ici 50 000), donc meme en cas de fuite, les mots de passe ne sont pas lisibles.

Jetons (tokens) : apres connexion, l'utilisateur recoit un jeton signe (HMAC) qui prouve son identite a chaque requete, sans renvoyer le mot de passe.

Docker : une "boite" qui contient l'application et tout ce dont elle a besoin (Python, librairies, modeles). Elle tourne pareil partout. On la deploie sur Hugging Face Spaces (16 Go de RAM gratuits, suffisant pour nos modeles).

Questions probables du jury + reponses

Q. Comment protegez-vous les donnees personnelles ?

Vérification d'identité avant toute action, mots de passe haches (PBKDF2), jetons signes, et le RAG filtre les billets par utilisateur pour qu'on ne voie jamais ceux d'un autre.

Q. Quelles limites de securite assumez-vous ?

La connexion Google n'est pas encore entierement verifiee cote serveur (on l'indique honnetement) et il manque une limitation de debit (rate limiting) contre le spam. Ce sont nos axes d'amelioration prioritaires.

Q. Pourquoi Hugging Face et pas un VPS ?

Gratuit, 16 Go de RAM (nos modeles d'IA tiennent), deploiement automatique a chaque mise a jour du code. Un VPS aurait ete payant et plus lourd a maintenir pour un projet etudiant.

3.6 Slide 10 : bilan, perspectives et echanges

Ce que tu dis (a l'oral)

"Pour conclure, le cahier des charges est couvert : recherche, reservation, assistant IA, RAG et reconnaissance d'entites. Nous assumons des limites : la connexion Google a durcir et des tests automatises a ajouter. Nos perspectives : ces tests, une supervision en production et le support de plus de langues. Merci de votre attention, nous sommes prêts a repondre a vos questions."

Questions probables du jury + reponses

Q. Qu'avez-vous trouve le plus difficile ?

Faire tenir des modeles d'IA (RAG, NER) sur un hebergement gratuit, et orchestrer proprement le chatbot pour qu'il agisse sur la base sans jamais halluciner d'information sur un billet.

Q. Si vous aviez plus de temps ?

Tests automatises (pytest), supervision/alertes, durcissement de l'authentification Google, et une vraie base vectorielle dediee.

4. Partie de Carl : back-end, API et base de donnees

Slide 7.

4.1 Slide 7 : back-end, actions reelles et donnees

Ce que tu dis (a l'oral)

"Cote back-end, j'ai construit l'API avec FastAPI : une trentaine de routes qui couvrent les comptes, la recherche de trajets, les billets, le chat et les reclamations, le tout sur 8 tables PostgreSQL. La securite des comptes repose sur des mots de passe chiffres et des jetons signes. Le point fort, ce sont les actions reelles : quand on cree un billet, l'API genere automatiquement un PDF et envoie un e-mail ; une modification ou une annulation est enregistree en base ; une reclamation recoit un numero de suivi. Enfin, les donnees sont realistes : 182 000 liaisons, et des prix et des places qui varient selon la saison, l'anticipation et le jour."

Ce que tu dois VRAIMENT comprendre

Une API REST, c'est un ensemble d'adresses (routes) que le front appelle : par exemple `POST /billets` pour creer un billet, `GET /trajets/search` pour chercher. Chaque route recoit des donnees, fait une operation, renvoie une reponse en JSON.

Les 8 tables : `routes` (les liaisons), `users` (utilisateurs), `trajets` (un trajet concret a une date), `billets`, `reclamations`, `chat_sessions` et `chat_messages` (les conversations), `admins`. Elles sont reliees par des cles etrangeres (un billet appartient a un utilisateur et a un trajet).

ORM SQLAlchemy : on ne ecrit pas du SQL a la main partout ; on manipule des objets Python (un `Billet`) que SQLAlchemy traduit en SQL. Plus sur, moins d'erreurs.

Prix dynamique : le prix de base est multiplie par des coefficients : saison (juillet plus cher), anticipation (reserver tot coute moins cher, au dernier moment plus cher), jour de la semaine. Les places se "remplissent" avec le temps de facon deterministe (calculee, reproductible).

Generation PDF + e-mail : a la creation d'un billet, le service genere un PDF (avec ReportLab) et l'envoi par e-mail (via Brevo). Ce sont de vraies actions.

Questions probables du jury + reponses

Q. Comment gerez-vous la recherche sur 182 000 lignes sans lenteur ?

Avec des index sur les colonnes de depart, d'arrivee et de type. Sans index, une recherche pourrait prendre plusieurs secondes ; avec, c'est quasi instantane.

Q. Qu'est-ce qu'une cle etrangere ?

Une colonne qui pointe vers la ligne d'une autre table. Par exemple `billets.user_id` pointe vers `users.id` : cela garantit qu'un billet appartient bien a un utilisateur existant.

Q. Comment migrez-vous de SQLite vers PostgreSQL ?

Un script (`migrate_to_postgres.py`) qui cree le schema sur PostgreSQL, vide la cible, puis copie les donnees table par table dans le bon ordre (a cause des cle etrangeres).

Q. Que se passe-t-il si l'envoi d'e-mail echoue ?

Le billet est quand meme cree en base ; l'e-mail est un complement, pas un bloquant. On pourrait ajouter une file d'attente pour reessayer.

Q. Pourquoi des jetons signes plutot que JWT standard ?

On a fait un format simple `id:expiration:signature` avec HMAC. C'est suffisant pour signer et verifier, sans dependance externe. JWT serait une evolution possible.

5. Partie de Noe : modeles de langage (NLP)

Slide 8.

5.1 Slide 8 : les modeles de langage

Ce que tu dis (a l'oral)

"Je me suis occupe des modeles de traitement du langage. Le premier role, c'est l'embedding : transformer le sens d'une phrase en une liste de nombres. On utilise le modele gte-small, qui produit des vecteurs de 384 dimensions et qui est multilingue ; cela permet de mesurer a quel point deux phrases sont proches. Ensuite, un cross-encoder reclasse les passages vraiment pertinents pour la question. Enfin, un modele de reconnaissance d'entites, CamemBERT, extrait automatiquement le nom, le prenom, la date et le numero de billet dans une phrase. Resultat : un dialogue plus precis et plus rapide, car l'utilisateur peut tout donner en une seule phrase."

Ce que tu dois VRAIMENT comprendre

Embedding (vectorisation) : un modele transforme "Combien coute un billet pour Rome ?" en un vecteur de 384 nombres. Une phrase au sens proche aura un vecteur proche. On mesure la proximite par la *similarite cosinus* (l'angle entre deux vecteurs).

gte-small : c'est notre *bi-encodeur*. "Small" car léger ; "384" c'est la taille du vecteur. Multilingue : marche en français comme en anglais.

Cross-encoder (ms-marco) : contrairement au bi-encodeur qui encode separement, lui prend la paire (question, passage) et donne un score de pertinence. Plus precis, donc on l'utilise pour reclasser les 10 candidats et garder les 3 meilleurs.

NER (Named Entity Recognition) : reconnaissance d'entites nommees. Le modele CamemBERT (un BERT entraine sur du français) lit une phrase et etiquette les mots : "Jean" = prenom, "Dupont" = nom, etc. On y ajoute des expressions regulieres (regex) pour les numeros de billet et les dates. Avantage : si l'utilisateur ecrit tout d'un coup, on evite de lui poser 4 questions.

Chargement paresseux (lazy loading) : les modeles sont charges en memoire seulement au premier usage, pour ne pas ralentir le demarrage.

Questions probables du jury + reponses

Q. C'est quoi BERT / CamemBERT ?

BERT est un modele de langage de Google qui "comprend" le contexte des mots. CamemBERT en est la version entraine sur du français. On l'utilise pour la reconnaissance d'entites.

Q. Pourquoi un modele de NER plutot que des regex partout ?

Les regex marchent pour des formats fixes (un numero de billet, une date). Mais un nom/-prenom est variable : le NER le reconnait dans n'importe quelle formulation.

Q. Comment evaluez-vous la qualite du RAG ?

On verifie que les passages remontes correspondent bien a la question (pertinence), et le reranker ameliore nettement le top 3 par rapport au bi-encodeur seul. On a teste sur des questions reelles (annulation, bagages).

Q. Qu'est-ce que la similarite cosinus ?

Une mesure entre -1 et 1 de l'angle entre deux vecteurs. Proche de 1 = sens tres proche. C'est ce qui classe les documents par ressemblance avec la question.

Q. Ces modeles tournent-ils en local ou via une API ?

En local, dans le conteneur (gte-small, ms-marco, CamemBERT sont telecharges). Seul Gemini, le modele qui redige, passe par une API.

6. Partie de Dhanoush : front-end, design et experience

Slide 9.

6.1 Slide 9 : interface, design et experience

Ce que tu dis (a l'oral)

"Cote interface, j'ai developpe le front-end en React, sous la forme d'une application en page unique : on navigue entre la recherche, la reservation, le profil et le chatbot sans recharger la page. L'interface communique directement avec l'API. Pour le design, on a choisi une identite visuelle orange, sobre, sans surcharge, avec des reponses de chat ou les sources sont cliquables. Et tout est pense pour fonctionner aussi bien sur mobile que sur ordinateur."

Ce que tu dois VRAIMENT comprendre

React : une bibliotheque pour construire des interfaces a base de *composants* (des briques reutilisables : un bouton, une carte de trajet, la fenetre de chat). Quand une donnee change, React met a jour seulement ce qui doit changer a l'ecran.

SPA (Single Page Application) : une seule page HTML ; la navigation se fait en JavaScript, sans rechargement. C'est fluide, comme une application.

Communication avec l'API : le front envoie des requetes (fetch) a l'API et affiche les reponses (trajets, messages du chat). Le jeton de connexion est garde dans le navigateur (localStorage) et envoie a chaque requete.

Rendu du chat : les reponses sont en Markdown ; les liens sources deviennent cliquables. Le composant affiche aussi quel outil a ete utilise (base, documentation, web) et des boutons de reponse rapide.

Responsive : la mise en page s'adapte a la taille de l'ecran (CSS), donc lisible sur telephone comme sur grand ecran.

Questions probables du jury + reponses

Q. Pourquoi React ?

C'est le standard pour des interfaces interactives, avec une logique en composants reutilisables et une mise a jour efficace de l'affichage.

Q. Comment le front sait-il qui est connecte ?

Apres connexion, on stocke un jeton dans le navigateur et on l'envoie dans l'en-tete de chaque requete. L'API verifie ce jeton.

Q. Comment gerez-vous l'affichage des reponses du chatbot ?

Le composant chat recoit le texte, l'affiche en Markdown (liens cliquables), montre les outils utilises et propose des reponses rapides pour guider l'utilisateur.

Q. Le site est-il accessible sur mobile ?

Oui, la mise en page est responsive : elle s'adapte automatiquement a la largeur de l'ecran.

7. Glossaire complet (a connaitre par tous)

API

Interface qui permet a deux programmes de se parler. Notre back-end expose une API que le front appelle.

FastAPI

Framework Python pour creer des API rapides et modernes.

REST

Style d'API basee sur des adresses (routes) et les verbes HTTP (GET, POST...).

Back-end / Front-end

L'arriere-plan (serveur, base, logique) / l'interface visible par l'utilisateur.

Base de donnees

Lieu de stockage organise des donnees. Ici PostgreSQL.

PostgreSQL / SQLite

Deux systemes de base de donnees ; SQLite = un fichier (dev), PostgreSQL = serveur robuste (prod).

ORM (SQLAlchemy)

Outil qui traduit des objets Python en requetes SQL.

Cle etrangere

Colonne qui relie une table a une autre.

Index

Structure qui accelere la recherche dans une table.

LLM

Large Language Model, grand modele de langage (ici Gemini) qui redige du texte.

Gemini 2.5 Flash

Le modele d'IA de Google qui genere nos reponses.

RAG

Retrieval-Augmented Generation : chercher des documents avant de generer la reponse.

Embedding / Vectorisation

Transformer un texte en vecteur de nombres representant son sens.

Vecteur / Dimension

Liste de nombres ; 384 dimensions = 384 nombres par texte.

Similarite cosinus

Mesure de proximite de sens entre deux vecteurs.

FAISS

Bibliotheque qui retrouve tres vite les vecteurs les plus proches.

Bi-encodeur (gte-small)

Encode question et documents separement : rapide.

Cross-encoder (ms-marco)

Compare question et passage ensemble : precis. Sert au reclassement (reranking).

Reranking

Reclasser les resultats pour remonter les plus pertinents.

Chunk

Morceau de document. On decoupe les textes pour les indexer.

NER

Named Entity Recognition : reperer nom, date, lieu... dans un texte.

BERT / CamemBERT

Modeles de comprehension du langage ; CamemBERT = version francaise.

Regex

Expression reguliere : motif pour reconnaitre un format (date, numero de billet).

Prompt

Le texte d'instruction envoye au modele d'IA.

Prompt injection

Tentative de manipuler l'IA pour contourner ses regles.

Hallucination

Quand une IA invente une information fausse. Le RAG aide a l'eviter.

Token / Jeton

Après connexion, preuve d'identité envoyée à chaque requête.

PBKDF2 / Hachage

Transformation irréversible d'un mot de passe pour le stocker en sécurité.

HMAC

Signature cryptographique qui garantit qu'un jeton n'a pas été falsifié.

OAuth / Google Sign-In

Connexion via un compte Google.

CORS

Règle de sécurité du navigateur sur les appels entre domaines différents.

Docker / Conteneur

"Boîte" qui embarque l'app et ses dépendances pour tourner partout pareil.

Hugging Face Spaces

Plateforme d'hébergement gratuit (16 Go RAM) où tourne notre conteneur.

Neon

Hébergeur gratuit de base PostgreSQL.

Variable d'environnement

Réglage (clef secrète...) injecté sans l'écrire dans le code.

React

Bibliothèque JavaScript pour construire l'interface en composants.

SPA

Single Page Application : navigation sans rechargement de page.

Markdown

Format de texte simple (liens, listes) utilisé dans les réponses du chat.

Responsive

Interface qui s'adapte à la taille de l'écran.

Open-Meteo / Frankfurter

APIs gratuites : meteo/heure locale et taux de change.

8. Questions transversales pieges (pour tous)

Questions probables du jury + reponses

Q. Quelle est la part de chacun dans le projet ?

Christian : IA conversationnelle, RAG, securite, deploiement. Carl : back-end, API, base de donnees. Noe : modeles NLP (embeddings, reranker, NER) et evaluation. Dhanoush : front-end, design, experience utilisateur. Chacun maitrise sa partie et comprend l'ensemble.

Q. Le chatbot peut-il se tromper ?

Oui, comme toute IA. C'est pourquoi on encadre les actions sensibles par des regles strictes (verification d'identite) et on ancre les reponses sur des sources reelles via le RAG.

Q. Qu'est-ce qui est "intelligent" exactement ?

La combinaison : comprehension du langage (Gemini, NER), recherche de la bonne information (RAG), et capacite a agir sur la base. Ce n'est pas qu'un repondeur, c'est un assistant qui execute.

Q. Est-ce reellement en production ?

Oui : application en ligne sur Hugging Face, base PostgreSQL chez Neon, code sur GitHub. On peut faire la demo en direct.

Q. Combien ca coute ?

Zero euro : hebergement, base et modeles locaux sont sur des offres gratuites. Seul Gemini consomme un quota gratuit d'API.

Q. Quelles sont vos limites et comment les corriger ?

Authentification Google a verifier cote serveur, manque de tests automatises et de limitation de debit. Plan : ajouter pytest, durcir l'OAuth, ajouter un rate limiting et de la supervision.

Repetez votre partie a voix haute. Relisez le glossaire la veille. Le jour J : respirez, regardez le jury, et racontez votre travail avec fierte.